

Enabling Privacy and Traceability in Supply Chains using Blockchain and Zero Knowledge Proofs

Shubham Sahai
IIT Kanpur
Kanpur, India
ssahai@cse.iitk.ac.in

Nitin Singh
IBM Research
Bangalore, India
nitisin1@in.ibm.com

Pankaj Dayama
IBM Research
Bangalore, India
pankajdayama@in.ibm.com

Abstract—In recent years there have been increased efforts to make supply chains *transparent* and *traceable* to better protect the end consumer's interests against counterfeiting, contamination, false claims, and inadequate processes. The actors in the supply chain can also leverage additional visibility to proactively tune their operations in response to upstream or downstream disruptions. Blockchain technology, with its promise of immutability, transparency, and provenance is a natural fit to supply chain problems. Several works have discussed the improved design of supply chains leveraging blockchain technology. There have been attempts to address some of the privacy concerns of participants on the blockchain network by using encryption schemes to hide confidential data and group signatures to hide the participant identities. But these do not handle the complexities of the supply chain, where items are constantly packaged, repackaged, and transformed before reaching the end consumer. Our work presents a blockchain-based model for the supply chain that provides privacy and traceability with efficient *contamination tracing* while accounting for the aforementioned complexities of the supply chain operations. The last property enables an end consumer to be convinced about a product to be unaffected by identified faulty upstream processes (contamination) without access to the entire history of the product. Our solution makes novel use of cryptographic tools such as zero-knowledge proofs and cryptographic accumulators to provide these guarantees. The choice of the cryptographic components is grounded in concrete efficiency, and we present an experimental evaluation of the implementation of our protocol on Hyperledger Fabric, a popular platform for enterprise blockchains.

Keywords—Privacy; Traceability; Supply Chain; Blockchain; Zero Knowledge;

I. INTRODUCTION

With the globalization of markets, the product supply chains are growing deeper and more complex involving multiple independent actors who may not trust each other with the data. Several activities are involved in the product's journey, starting with procuring raw materials through to final delivery of end product to the consumer. In this complex environment, two significant concerns have surfaced in the present supply chain management systems – lack of *transparency* and *traceability* – affecting all the stakeholders, be it governments, regulatory authorities, organizations or end consumers.

For instance, in the 2006 E.Coli outbreak in the US from fresh spinach [28], the FDA called out for bagged fresh spinach to be removed from the shelves and warned the US citizen regarding the same. The exact culprit turned out to be a 50-acre farm in California, but it took over a week to trace that

down. In that entire duration, the sale and the entire supply chain of organic spinach came to a halt, resulting in millions of dollars of losses to the farmers and stores.

From the consumer perspective, she is not aware of the fundamental aspects of the products, such as the origin of the raw materials being used, the manufacturing process, the chain of custody, conditions of delivery, storage, or transportation. This can result in reduced confidence and trust in the customers regarding the product's quality and authenticity. There have been cases in the past witnessing this behavior, for instance, in 2008, milk affected by melamine contamination due to unhealthy practices along the supply chain [25], resulted in a 33% reduction in consumption.

These types of incidents motivate the necessity of having transparent and traceable supply chains, and indeed efforts have been made in these directions [3], [17], [18], [27]. However, the distributed, and often international, nature of the supply chain made several of these efforts bottle-necked by the limitations of the centralized solutions in the past.

Related Work. In the last decade, blockchain has been increasingly gaining market traction as a foundational technology to prove end-to-end product provenance and implement track-and-trace of products through the supply chain in various sectors, including food, pharmaceutical, mining, etc. [1], [7], [13], [19], [21], [26]. Implementations of blockchain-based traceability solutions have been done in sectors such as cobalt mining [16] to produce lithium-ion batteries, where suppliers are required to abide by legal and responsible sourcing practices. In the food sector, [12] discusses the Tuna fish traceability to ensure authorized fishing compliance and proper cold chain management. IBM Food Trust [7] platform built in collaboration with Walmart has been deployed to track food products from farms to stores, with continuously growing adoption in these tough times due to COVID-19 [14]. In the pharmaceutical supply chain, Mediledger solution [23] deployed a zero-knowledge proof based solution that only provides proof of transaction while protecting the actual transaction data being shared. In 2016, [26] presented a solution for enabling traceability in agri-food supply chain in China. The solution combines the RFID and blockchain technologies to present a decentralized solution. In 2018, [19] proposes an ontology-driven blockchain solution for enabling traceability

in supply chains.

The primary issue with these blockchain-based solutions is the lack of complete protection of business-sensitive data. As the participants' competitors are present in the same supply chain, this lack of privacy is of grave concern. Recently, in 2019, [21] propose a privacy-preserving traceability solution for supply chains, that ensures unlinkability. They propose a decentralized traceability system, DECOUPLES, that uses blockchain and cryptographic primitives such as ring signatures and Schnorr signatures to provide a privacy-preserving solution. However, they do not consider complex supply chain operations such as splitting or merging of product lots or processing raw materials to produce a new product while modeling the supply chain.

Our Contributions. In this paper, we address the shortcomings and advance the previous work on privacy-preserving supply chains. The key features of our proposed solution include:

- (1) Privacy-preserving solution for tracking provenance using constant sized on-chain commitment for supply chain documents.
- (2) Efficient *contamination tracing* to determine if a product uses an identified contaminated lot of products in its processing (see Section III-C).
- (3) Our supply chain model is analogous to the real world and represents various supply chain operations via documents of appropriate types.
- (4) We consider common supply chain operations such as splitting and merging of product lots, which were not considered in prior work and take them into account while providing provenance and contamination.
- (5) Finally, the framework presented is modular, enabling the user to instantiate the framework with her choice of cryptographic accumulator and SNARK framework.

II. PRELIMINARIES

A. RSA Accumulators

A cryptographic accumulator [6] is a one-way function, that succinctly commits a set of values to a constant size digest. The commitments are hiding and binding, and support succinct proofs of membership, and often, non-membership of values in the accumulator. RSA accumulator [10] is a dynamic accumulator, which supports the addition of values to the accumulator, such that the cost of the operation is independent of the size of the accumulated set.

Let $\mathbb{G}$ be an RSA group of unknown order and $g \in \mathbb{G}$ be a generator. Then for a non-empty set S , its accumulator $\text{acc}(S)$ is given by $\text{acc}(S) := g^{\prod_{s \in S} s}$. Inserting a new value s' to the set S , requires updating the accumulator digest as $\text{acc}(S)^{s'}$.

RSA Accumulators support efficient non-membership proofs [20]. Notice that, given a prime x and a set of primes S , if $x \notin S$, then $\text{gcd}(x, \prod_{s \in S} s) = 1$. Consequently, the Bézout coefficients, i.e. the integers a, b satisfying $a.x + b.\prod_{s \in S} s = 1$ are a valid non-membership witness, and (g^a, b) is a succinct non-membership proof. We can verify the non-membership proof by checking if the following holds: $(g^a)^x \cdot (\text{acc}(S))^b = g$.

B. Zero Knowledge Proofs

We present an informal overview on zkSNARKs for arithmetic circuit satisfiability tailored to our requirements and refer the reader to [15], [22] for a more detailed and formal explanation.

zkSNARKs. Given an arithmetic circuit, C , a zkSNARK is defined as a triple of PPT algorithms $(\text{KGen}, \text{P}, \text{V})$ where:

- 1) $\text{KGen}(1^k, C) \rightarrow (\text{pk}, \text{vk})$. Given security parameter k and description of arithmetic circuit C , KGen generates the public parameters: *proving key* (pk) and the *verification key* (vk). This generation of keys constitutes the *one time trusted setup*.
- 2) $\text{P}(\text{pk}, x, w) \rightarrow \pi$. Given the proving key pk , the input x , and the witness w , P generates a publicly verifiable, non-interactive proof, π , for the circuit satisfiability.
- 3) $\text{V}(\text{vk}, \pi, x) \rightarrow \{0, 1\}$. Given the verification key vk , the proof π , and the input (x) , V outputs 1 if the verification is successful and 0 otherwise.

A zero-knowledge proof must satisfy three properties: *completeness* (an honest prover should be able to convince an honest verifier of a true statement), *soundness* (a cheating prover can convince an honest verifier with negligible probability) and *zero-knowledge* (no information apart from the veracity of the statement should be revealed).

III. PROBLEM SETTING

In this section, we describe our problem setting. First, we discuss modeling a typical supply-chain process as documents. Then, we define our supply chain model followed by our major objectives.

A. Modelling Supply Chain Processes

A product supply chain is a network of actors responsible for the complete journey of a product from the procurement of raw materials to the final delivery to the end consumer. The complex nature of a supply chain can be broken down to *processes*, such as – transferring goods from one party to another, acquiring raw material from a producer, or processing raw materials to produce finished products – which, analogous to supply chains in practice, we represent as documents of appropriate type, uniquely identifying the process, actors involved, and other specific details. We then model the supply chain as a directed graph, where each node represents a document, and each directed edge represents a relationship between the products identified by the corresponding documents. We discuss this in detail in Section III-B.

Documents in a supply chain. Supply chain documents have a well-defined structure associated with them, which can be encoded in a key-value format. The structure of the keys is pre-decided by the documents, which we refer to as the *template* of the document. Figure 1(a) gives a minimalistic template of a document, where *Type* denotes the document type, *InDocs* denotes the list of existing documents that a document refers to, and *DocInfo* is the actual content of the document specifying the details, such as item name, sending and receiving

<pre>Type: DocumentType InDocs: [D₁, D₂..., D_i] DocInfo: { Item: I, Sender: S, Recipient: R, Quantity: N }</pre>	<pre>Type: ProductShip InDocs: [D₁] DocInfo: { Item: CocoaBeans, Sender: CacaoWorld, Recipient: ChocoMagic, Quantity: 1000 }</pre>
(a)	(b)

Fig. 1: (a) A template of the document, (b) An example of a document of type `ProductShip`.

actors, quantity and other auxiliary information as needed. We emphasize here that the template of a document is pre-decided and publicly known. However, values corresponding to the template's keys are secret to the document's owner (sender) and receiver. We model the interactions in the supply chain by allowing the documents to refer to each other, via the `InDocs` field described above. An interaction between two participants P_1 and P_2 is abstracted by a document owned by P_2 referring to a document owned by P_1 .

Example 1: Let P_1 be a producer of cocoa beans and P_2 be a manufacturer of chocolates. In real world, if P_2 purchases some cocoa beans from P_1 , then there will be a purchase order corresponding to this interaction. We model this in our setting by two documents:

- 1) A document D_1 denoting that P_1 has produced k pounds of cocoa beans.
- 2) Another document D_2 denoting that there is a shipment from P_1 to P_2 containing $k' \leq k$ pounds of cocoa beans produced by P_1 . Figure 1(b) shows a minimalistic version of this document.

We relate the two documents via `InDocs` attribute, i.e., $D_1 \in D_2.\text{InDocs}$, and we say that document D_2 **consumes** document D_1 . At a later stage, if P_2 manufactures chocolate bars that use these cocoa beans, then there would be another document D_3 that consumes D_2 along with the documents corresponding to the other raw materials used. We show an example of such a workflow in Figure 2.

The **consumes** relation implicitly supports provenance checking and traceability, by tracing back the `InDocs` references starting from the desired product, eventually reaching the documents corresponding to the raw materials used. Next, we discuss the various types of documents that we use to model a typical supply chain business logic¹.

Document Types. We identify the following document types, abstracting the fundamental supply chain operations, that can be chained together to model complex business logic – (1) `Entry`: entry of a product in the supply chain, signifying the product as a primary commodity², (2) `Exit`: exit of a product from the supply chain system – when the product reaches

¹We do not intend to capture *all* the possible business logic. Instead, we present a generic framework that enables us to do so. The fundamental operations and document types presented can be expanded according to a given business requirement.

²A product at a minimally processed state for the system.

the shelves, (3) `Ship`: transferring product from sender to recipient, (4) `Merge`: combining the products in the referenced documents to form a single lot, (5) `Split`: dividing the products in the referenced document to separate lots, and (6) `Process`: process the products in the referenced documents to produce a new product.

B. Supply chain model

Building on the fundamental operations and their respective document types discussed above, we define the supply chain as follows.

Definition 1 (Supply Chain): A supply chain is a directed acyclic graph, $S := (D, E)$, where the set of vertices D are the documents of the types discussed above and $E \subset D \times D$ is the set of directed edges between these documents, such that, every edge $e := (d_1, d_2) \in E$ denotes that the document d_2 consumes d_1 .

Further, we require that the documents of type `Entry` do not consume any document, and another document can not consume the documents of type `Exit`. For a given document, D , **upstream** documents constitute all the documents from which D is reachable, and **downstream** documents are all the reachable documents from D . Then, provenance for a document can be established by tracing back all its upstream documents. However, we have still not addressed the privacy aspect of the supply chain. We discuss that in the next section, by giving the details of instantiating the supply chain model discussed above in a privacy-preserving manner using a blockchain.

C. Objectives

We now give an overview of the primary objectives of our system, deferring the detailed discussion to Section VI.

- 1) **Privacy**: Preserving the privacy of the business-sensitive data present in the documents.
- 2) **Unlinkability**: Preserving the privacy of the business relations, i.e., from the publicly available data, an observer should not be able to deduce the relationship between the participants of the supply chain.
- 3) **Traceability**: Providing complete provenance of the products.
- 4) **Contamination Tracing**: Determining if a product (final or intermediate) under consideration is contaminated – i.e. if it uses an identified contaminated lot of products in its processing.

IV. BLOCKCHAIN BASED INSTANTIATION

In this section, we present the blockchain-based instantiation of the above model of the supply chain in a privacy-preserving manner. We use Hyperledger Fabric as our choice of blockchain, inspired by the practicality and scalability offered by Fabric for business needs [2].

A. Blockchain Setup

In Fabric, the shared ledger is maintained by a network of participants known as *peers* and each participant on the network is called a *client*, which we denote by the set C . All clients have public identities (say using X.509 certificates),

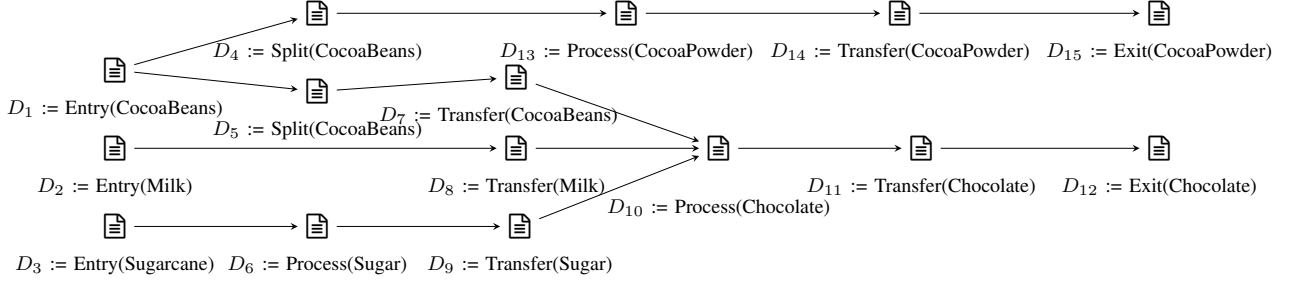


Fig. 2: Supply Chain Graph

Attribute	Purpose
Type	A string indicating the type of the document. Currently this is one among ProductEntry, ProductExit, ProductShip, ProductMerge, ProductSplit, ProductProcess.
DocHash	The hash of the contents of DocInfo structure for the document.
DocPrime	A unique prime associated with the document. This also serves as the ID of the document.
DocAccumulator	An accumulator value corresponding to the set of IDs of all the upstream documents of the given document.
InDocs	List (possibly empty) of IDs of the documents referenced in the current document.

TABLE I: Structure of the document asset in the blockchain

which are part of the network configuration and only the clients with configured identities can issue transactions on the blockchain network.

Anonymous Authentication. For unlinkability, our solution requires clients to be able to issue transactions anonymously. This is supported by newer versions of Fabric using *Identity Mixer* certificates, an implementation of Camenisch-Lysyanskaya anonymous credential system in [9], which use zero-knowledge proofs to prove knowledge of signature.

Document Asset. We represent the documents as on-chain assets of the blockchain. To ensure the privacy of the document's contents, we replace the DocInfo structure by a salted Merkle hash of its contents, which we refer to as DocHash. We discuss more about document hash in Section V-B. Additionally, we associate a unique prime integer and a set accumulator to each document asset. The prime integer is used as the **key** to store the document on the blockchain state database and use the accumulator for contamination tracing (discussed in Section V-A). The structure of the document asset is presented in Table I.

Trust Model. We assume all our adversaries are polynomially bounded. For anonymity and unlinkability guarantees we also assume that the adversary does not have access to network-level information such as originating IP addresses, or is unable to map those to participant identities. Further, we assume all participants have access to an incorruptible peer to interact with the blockchain network. Finally, we also assume that

the data on the blockchain can be trusted, by assuming that (i) trusted organizations (such as national cooperatives) introduce items to the supply chain, and (ii) a participant “consumes” a document that it received, only if the document hash matches the delivery it received off-chain, and (iii) for point of exit participants, we assume that they transfer the items to end customers, only if the document hash for the document describing the item is consistent with off-chain receiving document.

B. Blockchain Transactions

The supply chain operations identified in Section III are implemented as smart contracts. We now describe the transactions and the logic corresponding to the operations. All the transactions we consider have a payload of the form:

$$tx := (dt, InDocs[], OutDocs[], dh[], pcert[], *),$$

where, dt identifies the type of transaction, InDocs is a list of keys of documents to be consumed, OutDocs is a list of keys for indexing documents output by the transaction, dh is the list containing hashes of the DocInfo structures of the output documents, and pcert is the list of Pocklington's certificate (see Section V-A) to show that all keys in OutDocs are prime integers. The payload of a transaction additionally contains zero-knowledge proofs for establishing the well-formedness of the document or establishing the relationship between the contents of input/output documents. In the above, we use * to denote those proofs which depend on the type of transaction.

Next, we present the details of the different transaction types supported by our model.

Entry. A product *Entry* operation introduces an item on the supply chain. The corresponding supply chain transaction outputs a document doc_o describing the introduced item. The transaction tx_{entry} consists of the tuple: $tx_{entry} := (dt_{entry}, InDocs = [], OutDocs = [dp_o], dh = [h_o], pcert, \pi)$. Since this document does not have any previous references, InDocs is set to empty list ($[]$). The parameter dp_o denotes an unused prime to index the output document, while h_o denotes a hash of the contents of the output document. The transaction additionally contains Pocklington's certificate [24] pcert to show that dp_o is a prime integer, and a zero knowledge proof π to attest that $doc_o.Sender == doc_o.Recipient$ where $h_o = H(doc_o)$. Here, doc_o refers to the DocInfo structure of

the output document. If the proofs are valid, the smart contract updates the ledger L with mapping $L[\text{dp}_o] = \text{doc}_o$, where $\text{doc}_o := (\text{dt}, \text{dp}_o, \text{dAcc}_o, \text{dh}, \text{InDocs})$ and $\text{dAcc}_o = \text{acc}(\text{dp}_o)$.

Ship. A *Ship* operation refers to transfer of an item from a source organization (Sender) to a destination organization (Receiver). The transaction tx_{ship} is structured as: $\text{tx}_{\text{ship}} := (\text{dt}_{\text{ship}}, \text{InDocs}, \text{OutDocs}, \text{dh} = [h_o], \text{pcert}, \pi)$, where, InDocs consists of the key dp_i of input document, (say doc_i) while OutDocs consists of a unused prime integer dp_o (to be used as the key for the output document doc_o). The parameter pcert denotes the primality certificate for dp_o , while π denotes the zero knowledge proof for attesting: $(\text{doc}_i.\text{Recipient} == \text{doc}_o.\text{Sender}) \wedge (\text{doc}_i.\text{Item} == \text{doc}_o.\text{Item}) \wedge (\text{doc}_i.\text{Quantity} == \text{doc}_o.\text{Quantity})$.

Upon validation of the proofs, the smart contract computes the accumulator for the output document $\text{dAcc}_o = \text{acc}(\text{Set}(\text{doc}_i, \text{dp}_o))$, and updates $L[\text{dp}_o] = \text{doc}_o$ where $\text{doc}_o := (\text{dt}, \text{dp}_o, \text{dAcc}_o, h_o, \text{InDocs})$.

Split. A *Split* operation corresponds to several products derived from a common product. The corresponding transaction is therefore a single input multiple output transaction. For ease of presentation we consider two output documents $\text{doc}_{o,1}$ and $\text{doc}_{o,2}$ derived from an input document doc_i . The transaction payload tx_{split} consists of: $\text{tx}_{\text{split}} := (\text{dt}_{\text{split}}, \text{InDocs}, \text{OutDocs}, \text{dh}, \text{pcert}, \pi)$, where, $\text{InDocs} := [\text{dp}_i]$, $\text{OutDocs} := [\text{dp}_{o,1}, \text{dp}_{o,2}]$, $\text{dh} := [h_{o,1}, h_{o,2}]$ and $\text{pcert} := [\text{pcert}_1, \text{pcert}_2]$ where dp_i is a key for an existing document, $\text{dp}_{o,1}$ and $\text{dp}_{o,2}$ are unused prime integers with certificates pcert_1 and pcert_2 respectively. The zero knowledge proof π attests: $(\text{doc}_i.\text{Recipient} == \text{doc}_{o,1}.\text{Sender} == \text{doc}_{o,2}.\text{Sender}) \wedge (\text{doc}_i.\text{Quantity} == \text{doc}_{o,1}.\text{Quantity} + \text{doc}_{o,2}.\text{Quantity})$.

On successful verification of the proofs, the smart contract computes $\text{dAcc}_{o,j} := \text{acc}(\text{Set}(\text{doc}_i, \text{dp}_{o,j}))$ for $j = 1, 2$. Finally, the smart contract sets $L[\text{dp}_{o,j}] = \text{doc}_{o,j}$ where $\text{doc}_{o,j} = (\text{dt}, \text{dp}_{o,j}, \text{dAcc}_{o,j}, h_{o,j}, \text{InDocs})$ for $j = 1, 2$.

Merge. The merge operation corresponds to several discrete items received as part of different shipments being merged into a single packaging unit. The corresponding transaction is a multiple input single output transaction. The transaction tx_{merge} is structured as:

$$(\text{dt}_{\text{merge}}, \text{InDocs}, \text{OutDocs}, \text{dh} = [h_o], \text{dAcc}_{\text{out}}, \text{pcert}, \pi_1, \pi_2)$$

For simplicity of presentation, we consider the items of two documents $\text{doc}_{i,1}$ and $\text{doc}_{i,2}$ being merged to produce the output document doc_o , representing the merged lot. In this case, we have $\text{InDocs} = [\text{dp}_{i,1}, \text{dp}_{i,2}]$ and $\text{OutDocs} = [\text{dp}_o]$. The parameter pcert attests the primality of integer dp_o while π_1 is a zero knowledge proof for the following:

$$\begin{aligned} &(\text{doc}_{i,1}.\text{Recipient} == \text{doc}_{i,2}.\text{Recipient} == \text{doc}_o.\text{Sender}) \\ &\wedge (\text{doc}_{i,1}.\text{Quantity} + \text{doc}_{i,2}.\text{Quantity} == \text{doc}_o.\text{Quantity}) \end{aligned} \quad (1)$$

The merge transaction payload differs from those of *entry* and *shipment* transactions by including the accumulator value dAcc_o for the output document. In other transactions, the value of the accumulator can be calculated, however, in case of *Merge*, we wish dAcc_o to represent the union of set of upstream documents for each of the input documents, i.e, we wish to ensure: $\text{dAcc}_o := \text{acc}(U_{i,1} \cup U_{i,2} \cup \{\text{dp}_o\})$ where $U_{i,j}$ denotes the keys of all upstream documents of the document $\text{doc}_{i,j}$ for $j = 1, 2$. For most accumulators, the smart contract cannot compute dAcc_o without the knowledge of sets $U_{i,1}$ and $U_{i,2}$. Instead, we have the transaction initiator provide the value dAcc_o along with a zero knowledge proof π_2 showing that the value is correct. We present the details in the next section. The transaction is added to the ledger and on successful validation the smart contract updates the state database as $L[\text{dp}_o] = \text{doc}_o$, where $\text{doc}_o = (\text{dt}_{\text{merge}}, h_o, \text{dp}_o, \text{dAcc}_o, \text{InDocs})$.

Process. The transaction for *Process* operation is exactly like the one for *Merge*.

Exit. The transaction for the *Exit* operation is exactly like the one for *Ship*, with an extra requirement – document asset of type *exit*, can not be reference via InDocs in another transaction. This is to emphasize the fact that these documents refer to an end product for the supply chain.

V. CRYPTOGRAPHIC COMPONENTS

In this section, we give the details of the cryptographic components used to efficiently realize the overall design described in the previous sections.

A. Accumulators

We use practically efficient cryptographic accumulators based on RSA groups of unknown order, to succinctly store entire provenance of a document with it. For our solution, we choose a 1024 bit RSA modulus N (RSA-1024), whose factorization is unknown. Let g be an element of the multiplicative group $\mathbb{Z}_N^*$ with unknown order. Then for a set $S = \{p_1, \dots, p_k\}$ of odd primes, the accumulator of S , denoted by $\text{acc}(S)$ is defined as $\text{acc}(S) := g^{p_1 \cdots p_k} \bmod N$. Further, given an accumulator $\text{acc}(S)$ for a set S , the accumulator for the set $S' := S \cup \{p_{k+1}\}$ where $p_{k+1} \notin S$ is given by $\text{acc}(S') := (\text{acc}(S))^{p_{k+1}} \bmod N$. This is how the smart contract computes the accumulators for output documents in case of transactions involving single input document. Computing accumulators for outputs of *Merge* and *Process* transactions need to be addressed separately.

Proving Primality. Since the soundness of membership and non-membership proofs for RSA accumulators depend on the elements being primes; it is imperative that the smart contract checks each document key to be a prime. Efficiently checking numbers of large bit-size (≈ 500 in our case) in smart contracts using probabilistic tests such as Miller-Rabin can be expensive. To enable efficient verification, we have the prover generate primes that are “efficiently provable to be primes” based

on scheme in [11]. In [11], the certificate of primality for a prime n consists of k tuples $(n_1, a_1, r_1), \dots, (n_k, a_k, r_k)$ where $a_1 = r_1 = 0$, n_1 is a prime and $n_k = n$. As a consequence of Pocklington's theorem [24], a certificate is valid if the following conditions hold:

- 1) $\forall i \in [1, k]. r_i < n_i$ and $\forall i \in [2, n]. 2 \leq a_i < n_i$,
- 2) $\forall i \in [2, k]. n_{i+1} = 2r_i n_{i-1} + 1$,
- 3) $\forall i \in [2, n]. a^{n_i-1} \equiv 1 \pmod{n_i}$, and
- 4) $\forall i \in [2, n]. \gcd(a_i^{2r_i} - 1, n_i) = 1$.

For the case of 500 bit numbers, we have $k \approx 6$ (starting with a 20-bit prime) which leads to around 10 modular exponentiations of varying bit-lengths to verify the certificate. Asymptotically, the above verification costs $O(\ell)$ where ℓ is the bit-length of the prime being tested, whereas the Miller-Rabin test would cost $O(\lambda \ell^2)$ for error probability to be $\text{negl}(\lambda)$.

Computing Accumulator for Merge Transaction Outputs.

Suppose that a document doc_o is output of a merge transaction involving input documents $\text{doc}_{i,1}$ and $\text{doc}_{i,2}$ with accumulators $\text{dAcc}_{i,1}$ and $\text{dAcc}_{i,2}$ respectively. Let $S_{i,1}$ and $S_{i,2}$ be the set of documents represented by accumulators $\text{dAcc}_{i,1}$ and $\text{dAcc}_{i,2}$. In other words, we have $\text{dAcc}_{i,1} = g^{\Pi(S_{i,1})}$ and $\text{dAcc}_{i,2} = g^{\Pi(S_{i,2})}$, where $\Pi(S)$ denotes the product $\prod_{s \in S} s$. The provenance set for the output doc_o is given by $S_o = S_{i,1} \cup S_{i,2} \cup \{\text{dp}_o\}$ where dp_o is the prime integer corresponding to document doc_o . Computing the accumulator $\text{dAcc}_o = \text{acc}(S_o)$ in the smart contract would require knowledge of $\Pi(S_{i,1})$ and $\Pi(S_{i,2})$. However the bit lengths of those numbers scale linearly with the size of the sets $S_{i,1}$ and $S_{i,2}$ which is inefficient to store on the ledger. We circumvent this issue by having the transaction initiator supply the value dAcc_o as part of transaction payload along with a proof that dAcc_o is computed correctly. Concretely, the proof requires the prover to show knowledge of (X_1, X_2) such that $\text{dAcc}_{i,1} = g^{X_1}$, $\text{dAcc}_{i,2} = g^{X_2}$ and $\text{dAcc}_o = g^{X_1 \cdot X_2 \cdot \text{dp}_o^3}$. The preceding relation can be reduced to showing knowledge of $X_1 = X$ such that $u^X = v$ for $u = \text{dAcc}_{i,2}^{\text{dp}_o} \cdot g^{-1}$ and $v = \text{dAcc}_o \cdot \text{dAcc}_{i,1}^{-1}$, which is an instance of the relation $\mathcal{R}_{\text{PoKE}}$ in [8, Section 3.2], and can be proved using the protocol in there. For a transaction with m input documents, we need $m-1$ executions of the protocol. The verifier incurs 4 exponentiations for each protocol execution, and thus remains practical even for large values of m (see Section VII).

B. Zero knowledge proofs for well-formedness of the documents

Our solution ensures privacy of data by hiding the content represented by the `DocInfo` structure by replacing it with its hash-based commitment, discussed below. Nevertheless, the transaction validation requires certain predicates to be established over the document content, such as Sender-Recipient coupling or quantity conservation. We accomplish this by using Zero-Knowledge Succinct Arguments of Knowledge

³Note that the resulting accumulator may accumulate common elements more than once, but this does not impact membership/non-membership proofs

(SNARKs) proof combined with a hashing scheme with efficient selective opening, such as Merkle Hashes. While state of the art in SNARKs affords several possibilities, our choices are grounded in practical efficiency requirements: small on-chain storage and efficient on-chain verification. We discuss each of our components to realize the above module now.

Document Hash. Document specific content is represented by `DocInfo` structure, which contains sender and receiver identities, item code, quantity, and other auxiliary information (say location information, whether the produce is organic, etc.). We treat `DocInfo` as a list of values. The mapping between a key (such as Sender) and the position in the list is maintained as a public schema, called *template*. We commit to the document content by building a Merkle tree over the attribute values and computing the root hash. Merkle tree based commitment is suitable for our case, as most predicates only involve a few attributes of the document. We instantiate our Merkle tree using the SHA2 compression function $H: \{0, 1\}^{512} \rightarrow \{0, 1\}^{256}$. The choice of hash function allows us to pack 512 bits per leaf node. As an optimization, we pack four list elements (of 128 bits each) into every leaf node. Thus to support a `DocInfo` structure with 32 fields, we only need 8 leaf nodes, reducing the depth of the Merkle tree to 3. Further, we pack commonly used attributes in predicates, namely: Sender, Recipient, ItemCode, Quantity in a single leaf node. Thus, we only need to prove one authentication path to support predicates on these attributes.

Zero-Knowledge Proofs. Most of the efficient constructions of SNARKs prove statements expressed in the form of arithmetic circuit satisfiability. To invoke them, we first need to map predicates of interest to arithmetic circuit satisfiability instances. For on-chain verification, we use SNARKs in the *preprocessing* model. The protocols in this setting enable one-time preprocessing of an arithmetic circuit to generate a succinct description (*polylog* in circuit size) of the circuit. Subsequently, the verifier can verify the proofs against this circuit description. Since most preprocessing SNARKs also produce extremely short proofs (≈ 100 -300 bytes), they are incredibly efficient with respect to on-chain storage and verification costs. In this work, we use SNARKs with trusted setup [22], for superior proof sizes and efficient proof generation, although our solution can be instantiated using transparent setup protocols with some performance tradeoffs.

1) *Basic Gadgets for Zero-Knowledge Proofs.*: Though preprocessing SNARKs leads to efficient verifiers and small proofs, they lack the flexibility to support different types of computation: computations need to be known apriori and preprocessed. We work around this rigidity by preprocessing a set of *basic* circuits, which we call *gadgets*. Thereafter, by chaining proofs for these basic gadgets, we can prove predicates which permit decomposition into them, with efficient verification and small proof sizes. We instantiate following gadgets for our proofs:

1. $\text{MerklePath}(\text{rt}, i, (h_j)_{j=1}^4, (x_j)_{j=1}^4, (s_j)_{j=1}^4, (\text{path}_j)_{j=1}^4)$:

We use this gadget to prove that hash h_j commits to x_j for $1 \leq j \leq 4$, where x_1 - x_4 constitute the i^{th} leaf under the Merkle tree with root hash rt . More precisely, the gadget computes

$$rt := \text{ComputeRoot}(x_1 || \dots || x_4, i, (path_j)_{j=1}^4)$$

and $h_j = H(x_j || s_j)$ for $1 \leq j \leq 4$. In the above, ComputeRoot is a function that computes root of a Merkle tree of depth 4 with i^{th} leaf as $x_1 || \dots || x_4$, and $path$ as the list of partner nodes for the authentication path from i^{th} leaf to the root.

2. $\text{GadgetAdd}(hx, hy, hz; x, y, rx, ry, rz)$: We use this gadget to prove that hashes hx, hy and hz commit to values x, y and z where $z = x + y$. More precisely, the gadget computes $hx = H(x || rx)$, $hy = H(y || ry)$ and $hz = H(x + y || rz)$.
3. $\text{GadgetEq}(hx, hy; x, rx, ry)$: We use this gadget to show that hashes hx and hy commit to the same value. The gadget computes $hx = H(x || rx)$ and $hy = H(x || ry)$.
4. $\text{GadgetAnd}(hx, hy, hz; x, y, rx, ry, rz)$: We use this gadget to show that hashes hx, hy and hz commit to boolean values x, y, z such that $z = x \wedge y$.

We process the above gadgets to generate the following proving and verification key-pairs – $(pk_{\text{path}}, vk_{\text{path}})$, $(pk_{\text{add}}, vk_{\text{add}})$, $(pk_{\text{eq}}, vk_{\text{eq}})$, and $(pk_{\text{and}}, vk_{\text{and}})$.

Proofs for Predicates. We illustrate the proof generation and verification required for a merge transaction. Suppose the documents $\text{doc}_{i,1}$ and $\text{doc}_{i,2}$ are inputs to a merge transaction that outputs doc_o . We describe the proof generation for the predicate (1) in Section IV-B. Let $h_{i,1}$, $h_{i,2}$ and h_o be the hashes (Merkle hashes) of the DocInfo structures of the respective documents which are present on the ledger. We assume that the initial four attributes in the DocInfo structure are Sender, Recipient, ItemCode and Quantity, which according to our hashing scheme constitute the first leaf of the corresponding Merkle tree. The prover generates the proof as follows:

1. The prover individually commits to the four attributes of each of the documents. Let $H_{i,j}^S, H_{i,j}^R, H_{i,j}^I, H_{i,j}^Q$ denote hash commitments to the attributes Sender, Recipient, ItemCode and Quantity for $\text{doc}_{i,j}$, where $j = 1, 2$. Similarly, let $H_o^S, H_o^R, H_o^I, H_o^Q$ denote the hash commitments for these attributes for output document doc_o .
2. To prove correctness of the above hashed commitments, the prover uses the proving keys for MerklePath gadget to generate the following three proofs:
$$\pi_1 \leftarrow P(pk_{\text{path}}, h_{i,1}, 1, H_{i,1}^S, H_{i,1}^R, H_{i,1}^I, H_{i,1}^Q; \hat{w}_1),$$

$$\pi_2 \leftarrow P(pk_{\text{path}}, h_{i,2}, 1, H_{i,2}^S, H_{i,2}^R, H_{i,2}^I, H_{i,2}^Q; \hat{w}_2), \text{ and}$$

$$\pi_3 \leftarrow P(pk_{\text{path}}, h_o, 1, H_o^S, H_o^R, H_o^I, H_o^Q; \hat{w}_3),$$
where, $\hat{w}_1, \hat{w}_2$ and $\hat{w}_3$ denote the private inputs of the prover for the required gadget.
3. Next, the prover uses equality gadget GadgetEq to show that the sender in doc_o is the recipient in $\text{doc}_{i,1}$ and $\text{doc}_{i,2}$, which boils down to showing that $H_{i,1}^R, H_{i,2}^R$ and H_o^S commit to the same value. The prover generates the

following proofs: $\pi_4 \leftarrow P(pk_{\text{eq}}, H_{i,1}^R, H_o^S; *)$ and $\pi_5 \leftarrow P(pk_{\text{eq}}, H_{i,2}^R, H_o^S; *)$, where, we denote private inputs of the prover by $*$.

4. Finally, the prover uses GadgetAdd to establish the conservation of quantities in the merge by the following proof:
$$\pi_6 \leftarrow P(pk_{\text{add}}, H_{i,1}^Q, H_{i,2}^Q, H_o^Q; *).$$

The overall proof π consists of 12 hash commitments and 6 proofs $\pi_1, \dots, \pi_6$.

Verification of Proofs. The verifier decomposes the predicate into basic gadgets analogous to the prover and then verify proofs for each of the constituent gadgets. The proof for merge transaction discussed earlier can be verified against the public information by the verifier as:

1. Check the correctness of the hash commitments with respect to Merkle hashes $h_{i,1}, h_{i,2}$ and h_o of the documents as: $b_1 \leftarrow V(vk_{\text{path}}, h_{i,1}, H_{i,1}^S, H_{i,1}^R, H_{i,1}^I, H_{i,1}^Q)$, $b_2 \leftarrow V(vk_{\text{path}}, h_{i,2}, H_{i,2}^S, H_{i,2}^R, H_{i,2}^I, H_{i,2}^Q)$, $b_3 \leftarrow V(vk_{\text{path}}, h_o, H_o^S, H_o^R, H_o^I, H_o^Q)$.
2. Verify proofs for GadgetEq : $b_4 \leftarrow V(vk_{\text{eq}}, H_{i,1}^R, H_o^S)$ and $b_5 \leftarrow V(vk_{\text{eq}}, H_{i,2}^R, H_o^S)$.
3. Finally, verify the conservation of quantities. $b_6 \leftarrow V(vk_{\text{add}}, H_{i,1}^Q, H_{i,2}^Q, H_o^Q)$.

The verifier accepts if all the proofs are valid, i.e., $b_1, \dots, b_6$ are all True.

VI. SECURITY ANALYSIS

In this section, we summarize the security and privacy properties attained by our system and discuss how we achieve them. Most of the properties we claim, follow from the properties of the underlying cryptographic protocols we use. Let, $\text{SCP} := (\text{Entry}, \text{Exit}, \text{Ship}, \text{Merge}, \text{Process}, \text{Split})$ denote the smart contract protocols that interact with the shared ledger L . Recall that we say that a document D_2 consumes another document D_1 iff $D_1 \in D_2.\text{InDocs}$, which we denote by $D_1 \rightarrow D_2$. In the blockchain based instantiation, we represent this by $D_1 \rightarrow_L D_2$, to denote that a document asset D_1 is consumed by another asset D_2 in L if there is a transaction $\text{tx} \in L$ such that $D_1 \in \text{tx}.\text{InDocs}$ and $D_2 \in \text{tx}.\text{OutDocs}$. We naturally extend the consumes relation to the following representation $D_1 \rightarrow^* D_2$ to denote the existence of a hierarchy of documents $D_{i1}, D_{i2}, \dots, D_{iN}$ such that, $D_1 \rightarrow D_{i1} \rightarrow D_{i2} \rightarrow \dots \rightarrow D_{iN} \rightarrow D_2$. The analogous counterpart for the blockchain based instantiation is $D_1 \rightarrow_L^* D_2$.

A. Security of SCP

We prove the security of SCP against a PPT adversary $\mathcal{A}$ having visibility of the shared ledger L , and can interact with the ledger by submitting transactions according to the protocol SCP, i.e., the adversary can see the transaction payload recorded on the ledger, but the private inputs used to create the transaction are hidden from the adversary. We call such adversary an *interactive adversary*. We assume that a transaction tx is recorded on the ledger as: $\text{tx} := (\text{dt}, \text{dp}, \text{dAcc}, \text{InDocs}, \text{OutDocs}, \text{dh}, \text{pcert}, \pi^*, \text{acert})$, where, π^* denotes the proofs furnished by the transaction

initiator for validation by the smart contract and *acert* denotes the anonymous credential, such as a Camenisch-Lysyanskaya signature [9]. The rest of the parameters are as discussed in Section IV-B.

Definition 2 (Unlinkability): We say that the protocol SCP satisfies *unlinkability* if it is infeasible for an interactive adversary $\mathcal{A}$ to determine if two different transactions were invoked by the same participant.

Definition 3 (Recipient Hiding): We say that the protocol SCP satisfies *recipient hiding* if it is infeasible for an interactive adversary $\mathcal{A}$ to distinguish between transactions with two different recipients.

We mention that the above properties of *unlinkability* and *recipient hiding* follow from the notion of *Ledger Indistinguishability*, introduced in [5], which captures the fact that an adaptive adversary interacting with a ledger L learns nothing beyond the public information on the ledger. We use cryptographic tools similar to those in [5] – hiding commitments (hashes with random salt) to hide private data, zero-knowledge proofs to prove assertions on the private data and zero-knowledge proof based anonymous credential system [9] for signing transactions. This enables us to formulate games on the lines of [5] and the detailed proofs there can also be adapted to our setting.

Our next notion of *traceability* states that complete provenance of each document asset on the ledger can be verified using the transaction records.

Definition 4 (Traceability): We say that the protocol SCP satisfies *traceability* if for any two documents D_1 and D_2 , $D_1 \rightarrow D_2$ if and only if $D_1 \rightarrow_L D_2$.

Finally, we define the notion of *contamination tracing* for our protocol, which allows a party to determine if documents D_1 and D_2 are linked by a series of supply chain operations, without access to intermediate documents.

Definition 5 (Contamination Tracing): We say that the protocol SCP satisfies *contamination tracing* if there exist PPT algorithms Gen (setup), $\text{P}_\in$ (membership proof generation), $\text{P}_\notin$ (non-membership proof generation), $\text{V}_\in$ (membership proof verification), and $\text{V}_\notin$ (non-membership proof verification) such that for any $D_1, D_2 \in L$:

(i) If $D_1 \rightarrow^* D_2$, there exists witness w such that:

$$\Pr \left[\text{V}_\in(\sigma, D_1, D_2, \pi) = 1 \mid \begin{array}{l} \sigma \leftarrow \text{Gen}(1^k) \\ \pi \leftarrow \text{P}_\in(\sigma, D_1, D_2, w) \end{array} \right] = 1$$

(ii) If $D_1 \not\rightarrow^* D_2$, for any PPT adversary $\mathcal{A}$:

$$\Pr \left[\text{V}_\in(\sigma, D_1, D_2, \pi) = 1 \mid \begin{array}{l} \sigma \leftarrow \text{Gen}(1^k) \\ \pi \leftarrow \mathcal{A}(\sigma, D_1, D_2) \end{array} \right] = \text{negl}(k)$$

(iii) If $D_1 \not\rightarrow^* D_2$, there exists witness w such that:

$$\Pr \left[\text{V}_\notin(\sigma, D_1, D_2, \pi) = 1 \mid \begin{array}{l} \sigma \leftarrow \text{Gen}(1^k) \\ \pi \leftarrow \text{P}_\notin(\sigma, D_1, D_2, w) \end{array} \right] = 1$$

(iv) If $D_1 \rightarrow^* D_2$, for any PPT adversary $\mathcal{A}$:

$$\Pr \left[\text{V}_\notin(\sigma, D_1, D_2, \pi) = 1 \mid \begin{array}{l} \sigma \leftarrow \text{Gen}(1^k) \\ \pi \leftarrow \mathcal{A}(\sigma, D_1, D_2) \end{array} \right] = \text{negl}(k)$$

The definition essentially captures that linkage or absence of linkage between any two documents can be proven and verified using the document records on the ledger. In this case, the verifier trusts the records from the ledger but does not have unlimited access to it.

Our protocol satisfies the above notion of contamination tracing by using cryptographic accumulators. As described in Section V and V-A, each document contains accumulator for the set of all its upstream documents. Thus, the linkage/non-linkage between two documents reduces to membership/non-membership of one document in the upstream set of the other. The properties follow from the soundness and completeness properties of the RSA accumulators.

VII. EXPERIMENTAL RESULTS

In this section we provide experimental evaluation of our proposed system.

Computation Setup. For our experiments, we used Hyperledger Fabric v2.0 as the underlying blockchain, deployed on Docker version 19.03.11. The chaincode and algorithms for generating provable primes and RSA accumulators were implemented in Golang. We used `libsnaark` library [4] to build the circuits for basic gadgets discussed in Section V-B and for generating and verifying SNARK proofs. The experiments were run on an Intel i7-4770 CPU @ 3.40GHz with 8 cores and 32 GB RAM running Ubuntu 18.04.

A. Experimental Evaluation

We evaluate our system from two standpoints. First, we evaluate the timings of the individual cryptographic algorithms involved in generating and verifying the transaction payloads on a blockchain. Then we analyze the performance of the complete system, by evaluating the time needed for the generation of chaincode payloads and chaincode execution.

Evaluation of cryptographic components. Table II presents the time taken and the parameters involved for executing the key algorithms used for maintaining the *RSA accumulator* – $\text{GenPrime}(B)$ (Generates a random prime of B bits along with a primality certificate, `pcert`), $\text{VerifyPrime}(p, \text{pcert})$ (Checks whether p is a prime given the primality certificate `pcert`), $\text{UpdateAcc}(A, p)$ (Computes the updated value of accumulator A by adding a new prime element p), $\text{CombineAcc}(A_1, A_2)$ (Computes the value of accumulator corresponding to union of sets represented by A_1 and A_2), and $\text{VerifyAcc}(A, A_1, A_2, \dots, A_i)$ (Verifies whether accumulator A represents the union of sets represented by accumulators $A_1, A_2, \dots, A_i$).

Contamination Tracing. One of our protocol's key features is to efficiently prove to an end-customer that given a document for a final product F , it does not involve an identified upstream document D . Here the customer trusts blockchain records for D and F , but does not have access to the complete supply chain graph. As each document record maintains an accumulator for the set of all its upstream documents, the

Algorithm	GenPrime	VerifyPrime	UpdateAcc	CombineAcc	VerifyAcc
Parameters	$B = 500$		-	$ S = 100$	$m = 2$
Time(ms)	15	0.32	0.25	25	1.05

TABLE II: Evaluation of algorithms maintaining the RSA accumulator. The size of RSA modulus is 1024 bits, $|S|$ denotes the size of the sets combined, and m denotes the number of sets considered for union in `VerifyAcc`.

Algorithms	Proof Generation			Proof Verification		
	MerklePath	G-Add	G-Eq	MerklePath	G-Add	G-Eq
TimeST(ms)	16000	4950	3200	35	35	35
TimeMT(ms)	5200	1600	1000	48	48	48

TABLE III: Evaluation of proof generation and proof verification of SNARKs gadgets and Merkle Paths. The performance of `GadgetAnd` is identical to that of `GadgetAdd` and is omitted in the table above.

Algorithms	Non-Membership Proof Generation				Non-Membership Proof Verification			
Accumulated Set size	100	250	500	1000	100	250	500	1000
Time(ms)	39	108	156	308	0.54	0.54	0.54	0.54

TABLE IV: Evaluation of non-membership proof generation and proof verification for RSA Accumulator.

Algorithms	Transactions				
	Entry	Ship	Merge	Split	Exit
GetPrime	1	1	1	$m = 2$	1
CombineAcc	0	0	0	$m - 1 = 1$	0
ProveMerklePath	1	2	$m + 1 = 3$	$m + 1 = 3$	1
ProveAdd	0	0	$m - 1 = 1$	$m - 1 = 1$	0
ProveEq	1	3	$m = 2$	$m = 2$	1
Size (Bytes)	640	1216	1404	1600	640
TimeST(s)	21	41.6	59.4	59.4	21.2
TimeMT(s)	6.2	13.4	19.2	19.2	6.3

TABLE V: Evaluation of the size, time taken and the algorithms used in generation of transaction payload. These computations are executed off-chain. The time taken by *Process* is identical to *Merge*, hence omitted here.

Algorithms	Transactions				
	Entry	Ship	Merge	Split	Exit
VerifyPrime	1	1	1	$m = 2$	1
UpdateAcc	1	1	0	$m = 2$	1
VerifyAcc	0	0	$m - 1 = 1$	0	0
VerifyMerklePath	1	2	$m + 1 = 3$	$m + 1 = 3$	1
VerifyAdd	0	0	$m - 1 = 1$	$m - 1 = 1$	0
VerifyEq	1	3	$m = 2$	$m = 2$	1
Time(ms)	71	176	211	212	70

TABLE VI: Evaluation of the time taken and the algorithms used in execution of chaincode transactions. The time taken by *Process* is identical to *Merge*, hence omitted here.

preceding proof reduces to proof of non-membership of the key for D in the accumulator $\text{acc}(F)$. Table IV summarizes the performance of generation and verification of such non-membership proofs for different values of accumulated set sizes. We observe that our solution has very little overhead for practically relevant values of the size of the accumulated set ($|\text{acc}(F)|$) for non-membership proof generation. Proof

verification on the other hand is extremely fast and takes constant time.

Next, we analyze the performance of the SNARK gadgets presented in Section V-B. We report the proof generation and proof verification time for each of the gadgets in a single-threaded and multi-threaded (8 threads) setting in Table III.

Evaluation of the chaincode transactions. The generation and execution of the chaincode transactions, as discussed in Section IV-A, consist of an intermix of the algorithms discussed above. We evaluate the time taken for the generation and execution of each transaction, and also present in Table V and VI how many times each of the algorithms discussed above are invoked in these transactions.

Table V presents the size and the time taken for the generation of the transaction payload. The computations are performed at the client-side, making several seconds taken for the generation of payloads affordable in practice. The generated transaction payload is then sent to the blockchain to invoke the corresponding chaincode function and execute the smart contract.

Finally, Table VI presents the execution time for each chaincode transaction, on-chain. Every chaincode execution requires verifying a set of proofs and updating the accumulator values as discussed in Section IV-A. We would like to emphasize here that the fast on-chain execution time of the above transactions and small size of the payload makes the proposed solution adaptable in a real-world setting.

As discussed earlier, our choice of using the SNARKs with trusted setup [22] in the presented solution is motivated by the practical concerns – the faster on-chain verification time and small proof size. However, our solution can also be instantiated using transparent setup protocols with performance tradeoffs.

VIII. CONCLUSION

In this work, we presented a practical system to enable traceability in a privacy-preserving manner on the supply chain. Our system models the supply chain analogous to the real-world, and addresses problems such as privacy of data and business relations, as well as contamination tracing which has eluded the systems in the past. Further, we support traceability while considering the supply chain operations such as splitting and merging of the product lots and processing of raw materials to finished products, which has again not been addressed by the solutions in the past. In concrete realization of our scheme, we have chosen commonly supported and standards-compliant hash functions, and efficient constructions of SNARK to further underline the practical applicability of our system. Finally, the system intends to serve as a generic framework, which can be instantiated with any hash function and any SNARK primitives in a trusted setup or transparent setup model.

REFERENCES

- [1] Saveen A Abeyratne and Radmehr P Monfared. Blockchain ready manufacturing supply chain using distributed ledger. *International Journal of Research in Engineering and Technology*, 5(9):1–10, 2016.
- [2] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–15, 2018.
- [3] Alessio Bechini, Mario GCA Cimino, Francesco Marcelloni, and Andrea Tomasi. Patterns and technologies for enabling supply chain traceability through collaborative e-business. *Information and Software Technology*, 50(4):342–359, 2008.
- [4] E. Ben-Sasson, A. Chiesa, D. Genkin, S. Kfir, E. Tromer, M. S. L. Virza, and others external contributors. libsnark: A c++ library of zk snark proofs, 2017.
- [5] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy, S&P 2014.*, pages 459–474, 2014.
- [6] Josh Benaloh and Michael De Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 274–285, 1993.
- [7] IBM Blockchain. Ibm food trust, 2020.
- [8] Dan Boneh, Benedikt Bünz, and Ben Fisch. Batching techniques for accumulators with applications to iops and stateless blockchains. In *Advances in Cryptology – CRYPTO 2019*, pages 561–586, 2019.
- [9] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology – EUROCRYPT*, pages 93–118, 2001.
- [10] Jan Camenisch and Anna Lysyanskaya. Dynamic accumulators and application to efficient revocation of anonymous credentials. In *Annual International Cryptology Conference*, pages 61–76, 2002.
- [11] Christophe Clavier, Benoit Feix, Loïc Thierry, and Pascal Paillier. Generating provable primes efficiently on embedded devices. In *Public Key Cryptography – PKC 2012*, pages 372–389, 2012.
- [12] Bubba Cook and WN Zealand. Blockchain: Transforming the seafood supply chain.
- [13] Mourad El Maouchi, Oguzhan Ersoy, Zekeriya Erkin, et al. Trade: A transparent, decentralized traceability system for the supply chain. In *Proceedings of 1st ERCIM Blockchain Workshop 2018*. European Society for Socially Embedded Technologies (EUSSET), 2018.
- [14] Forbes. Ibm blockchain is growing in the food industry during covid-19, 2020.
- [15] S Goldwasser, S Micali, and C Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC, pages 291–304, 1985.
- [16] Gabriella M Hastig and Manmohan S Sodhi. Blockchain for supply chain traceability: Business requirements and critical success factors. *Production and Operations Management*, 29(4):935–954, 2020.
- [17] Yong-Shin Kang and Yong-Han Lee. Development of generic rfid traceability services. *Computers in industry*, 64(5):609–623, 2013.
- [18] Thomas Kelepouris, Katerina Pramatar, and Georgios Doukidis. Rfid-enabled traceability in the food supply chain. *Industrial Management & data systems*, 2007.
- [19] Henry M Kim and Marek Laskowski. Toward an ontology-driven blockchain design for supply-chain provenance. *Intelligent Systems in Accounting, Finance and Management*, 25(1):18–27, 2018.
- [20] Jiangtao Li, Ninghui Li, and Rui Xue. Universal accumulators with efficient nonmembership proofs. In *International Conference on Applied Cryptography and Network Security*, pages 253–269. Springer, 2007.
- [21] Mourad El Maouchi, Oguzhan Ersoy, and Zekeriya Erkin. Decouples: a decentralized, unlinkable and privacy-preserving traceability system for the supply chain. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, pages 364–373, 2019.
- [22] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE, 2013.
- [23] Teijo Peltoniemi and Jarkko Ihalainen. Evaluating blockchain for the governance of the plasma derivatives supply chain: How distributed ledger technology can mitigate plasma supply chain risks. *Blockchain in Healthcare Today*, 2:10–30953, 2019.
- [24] Henry C Pocklington. The determination of the prime or composite nature of large numbers by fermat’s theorem. *Proc. Cambridge Philosophical Society*, 1914, 18:29–30, 1914.
- [25] Guanghua Qiao, Ting Guo, and KK Klein. Melamine and other food safety and health scares in china: Comparing households with and without young children. *Food Control*, 26(2):378–386, 2012.
- [26] Feng Tian. An agri-food supply chain traceability system for china based on rfid & blockchain technology. In *2016 13th international conference on service systems and service management (ICSSSM)*, pages 1–6, 2016.
- [27] Jack GAJ Van Der Vorst, Seth-Oscar Tromp, and Durk-Jouke van der Zee. Simulation modelling for food supply chain redesign; integrated decision making on product quality, sustainability and logistics. *International Journal of Production Research*, 47(23):6611–6631, 2009.
- [28] Wikipedia. 2006 north american e. coli o157:h7 outbreak in spinach, 2020.